

RIX 2024 - Batch 1- Selenium Practical Assessment

12576 Nisanth Saravanan

Online Courier Management System

An online courier management system is a software application that is used to manage the process of sending and tracking packages and documents through a courier service. This typically includes functions such as creating shipping labels, tracking packages, and generating reports. This project is a useful tool for businesses and individuals who need to send packages and documents through a courier service on a regular basis.

W3H Analysis

Project Title: Online Courier Management System	
What?	How?
1. What are the necessary modules? A: Modules Needed: 1) Login and Registration. 2) Admin Dashboard. 3) User Dashboard.	Login and Registration Module: 1) Register User: Method 1: Use Simple Form Page. Method 2: Use a Modular Form Page. Method 3: Use Google OAuth and other 3rd party services.
2. What are the necessary functionalities of Login and Registration Modules? A: Login and Registration Module: 1) Register User. 2) Login as User. 3) Login as Admin.	2) Login as User: Method 1: Use Simple Form. Method 2: Use a Modular Form. Method 3: Use 3rd party services.
3. What are the necessary functionalities of Admin Dashboard? A: Admin Dashboard: 1) Perform CRUD Operation with Shipments. 2) Filter Shipments.	3) Login as Admin: Method 1: Hard Code the Admin accounts. Method 2: Have an Admin Login Page. Method 3: Have a separate Register + Login for admins.
4. What are the necessary functionalities of User Dashboard? A: User Dashboard: 1) Filter Shipments.	Admin Dashboard: 1) Perform CRUD Operation with Shipments: Method 1: Use Simple Forms and Buttons to perform CRUD. Method 2: Have separate pages for each operation Method 3: Use Modals.
5. What are the tables needed? A: Tables Required:	

- Shipments
- Users

6. What are the fields of user table?

A: Fields of Users table:

- Id (Primary Key) - int
- Name – string
- Password – string
- Role – string

7. What are the fields of the Shipment table?

A: Fields of Shipment Table:

- Id (Primary Key) - int
- Name – string
- Type – string
- Address – string

8. What are the two values of the Role Field in the User table?

A: Admin Role

User Role

9. What are the must have dependencies?

A: Dependencies:

- axios
- react-router-dom
- toastr

10. What are the validations in the application?

A: Overall validations list:

- Frontend simple JS Validations.
- Server Side validations.
- Form Validations.

2) Filter Shipments:

Method 1: Use a simple search bar.

Method 2: Have tables and sorting buttons.

Method 3: Have a separate page to pick out exact results.

User Dashboard:

1) Filter Shipments:

Method 1: Use a simple search bar.

Method 2: Have tables and sorting buttons.

Method 3: Have a separate page to pick out exact results.

Login and Registration Module:

1) Register User:

Method 1: Use Simple Form Page.

A: Since the overall scale of the application is small and data needed is rather minimal, it's wise to stick with traditional forms.

2) Login as User:

Method 1: Use Simple Form.

A: Since the overall scale of the application is small and data needed is rather minimal, it's wise to stick with traditional forms.

3) Login as Admin:

Method 1: Hard Code the Admin accounts.

A: It's best to hard code admin users to ensure security and have only one or two admin users in total.

Admin Dashboard:

1) Perform CRUD Operation with Shipments:

Method 1: Use Simple Forms and Buttons to perform CRUD.

A: Since the overall scale of the application is small and data needed is rather minimal, it's wise to stick with traditional forms.

2) Filter Shipments:

Method 1: Use a simple search bar.

A: Effective as it may, the simple input field can be directly mapped with the

Login and Registration Module:

1) Register User:

Method 2: Use a Modular Form Page.

A: Increases the complexity of the app, and also the fact that the number of needed inputs is rather minimal.

Method 3: Use Google OAuth and other 3rd party services.

A: Valid method but due to the scale of the application, this method is seemingly overkill.

2) Login as User:

Method 2: Use a Modular Form.

A: Increases the complexity of the app, and also the fact that the number of needed inputs is rather minimal.

Method 3: Use 3rd party services.

A: Valid method but due to the scale of the application, this method is seemingly overkill.

3) Login as Admin:

Method 2: Have an Admin Login Page.

A: Best not to have an admin login page distinctively when the primary user of the app is regular customers.

Method 3: Have a separate Register + Login for admins.

A: This increases the number of pages to be developed and also leaves open a vulnerable endpoint for someone to register as administrator.

shipments that are being displayed.

User Dashboard:

1) Filter Shipments:

Method 1: Use a simple search bar.

A: Effective as it may, the simple input field can be directly mapped with the shipments that are being displayed.

Admin Dashboard:

1) Perform CRUD Operation with Shipments:

Method 2: Have separate pages for each operation.

A: Since the admin needs all information in one place and also that the number of operations they can do is limited, it is best to have a single page control centre type.

Method 3: Use Modals.

A: Seeing that the number of input fields are minimal, and asking for user input using modals is seemingly overkill.

2) Filter Shipments:

Method 2: Have tables and sorting buttons.

A: This method is also viable but not selected due to the slight increase in the number of code needed for its execution.

Method 3: Have a separate page to pick out exact results.

A: Having the all the data displayed in one page and search functionality in another is not idle. Hence, this method is rejected.

User Dashboard:

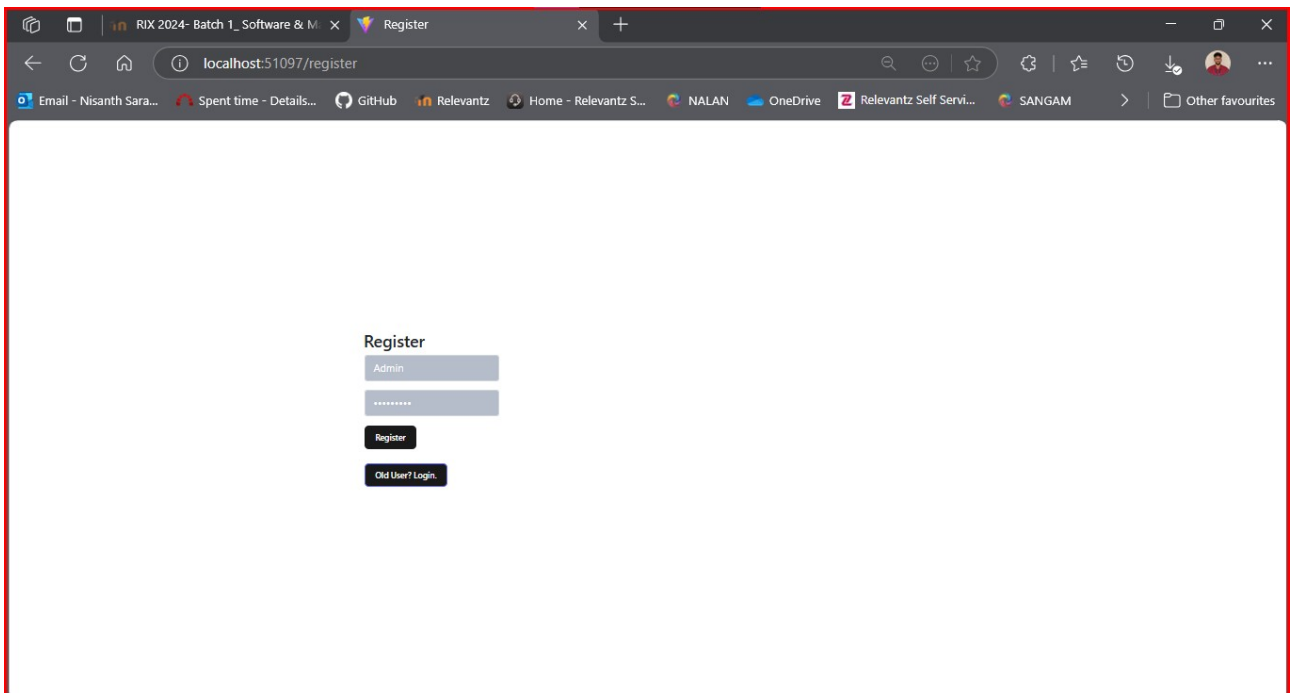
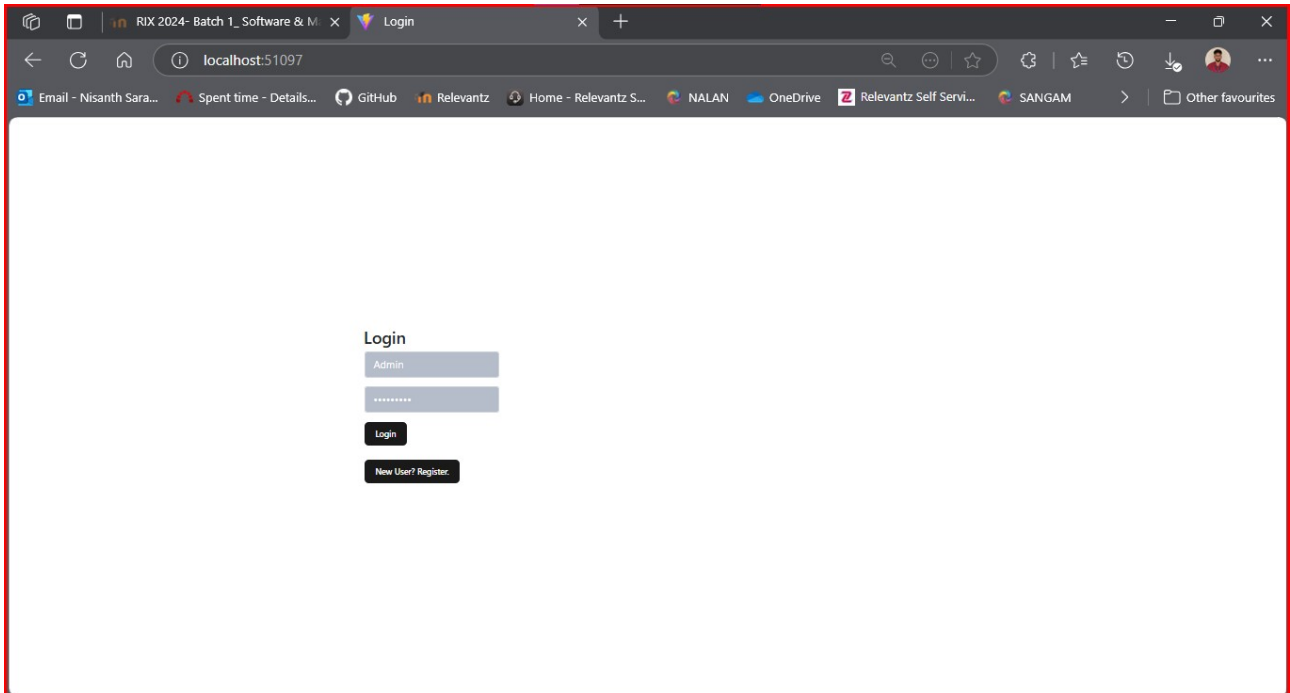
1) Filter Shipments:

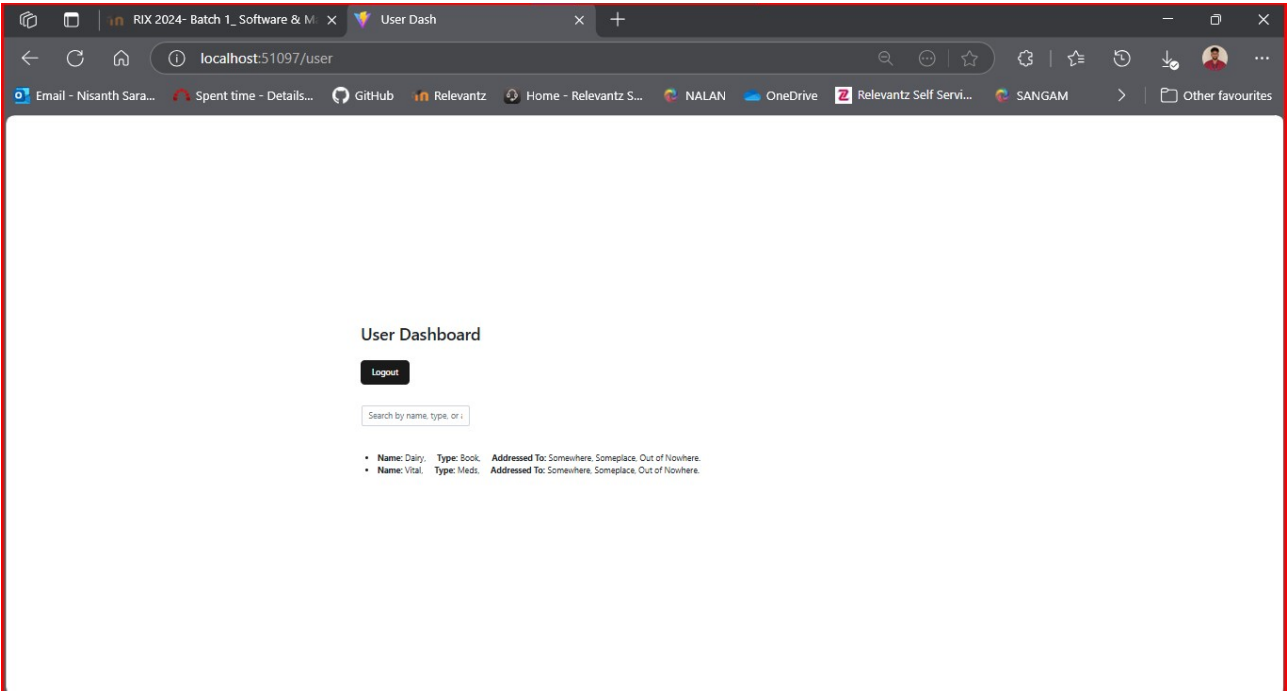
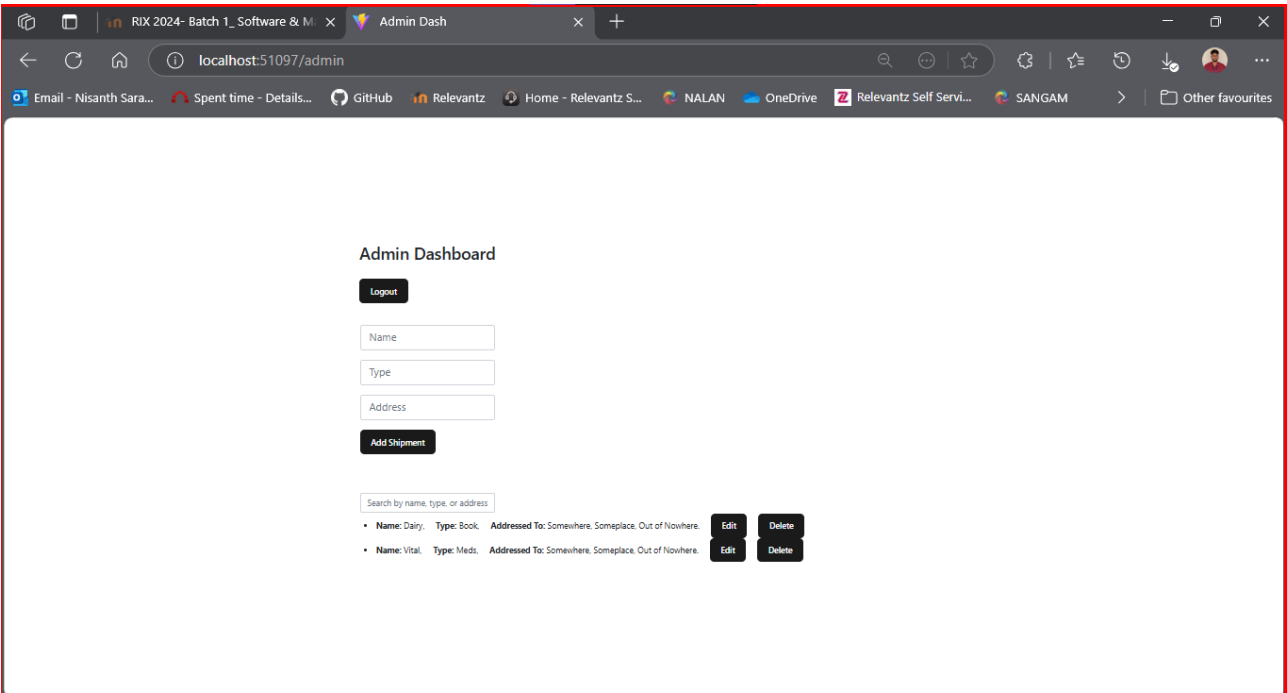
Method 2: Have tables and sorting buttons.

A: This method is also viable but not selected due to the slight increase in the

	number of code needed for its execution. Method 3: Have a separate page to pick out exact results. A: Having the all the data displayed in one page and search functionality in another is not idle. Hence, this method is rejected.
Why?	Why Not?

Output Screenshots





Testing:

AdminUserSeleniumTester

Project name :	Online Courier Management System
Module Name :	Admin User / Admin Dashboard
Reference Document :	<u>AdminUserSeleniumTester</u>
Created by :	Nisanth Saravanan
Date of creation :	13-08-2025
Date of review :	

SubmitEmptyRegisterForm	Admin tries to submit an empty register form.	Registration Validations	Navigate to Register Page	No Logged In User.	Admin is alerted that all fields are necessary.	Admin is alerted that all fields are necessary.	Pass
			Click on the "Register" Button.				
SubmitEmptyLoginForm	Admin tries to submit an empty register form.	Registration Validations	Navigate to Login Page	No Logged In User.	Admin is alerted that all fields are necessary.	Admin is alerted that all fields are necessary.	Pass
			Click on the "Login" Button.				
SubmitInvalidCredentials	Admin tried to login but credentials are invalid.	Login Failure due to Credentials.	Navigate to the Login Page.	No User Logged in.	Admin is alerted that credentials are invalid.	Admin is alerted that credentials are invalid.	Pass
			Enter credentials.	Invalid Credentials.			
			Click on the "Login" Button.				
LoginAsAdmin	Admin logs in successfully.	Login Success	Navigate to the Login Page.	No User Logged in.	Admin is redirected to Admin Dashboard.	Admin is redirected to Admin Dashboard.	Pass
			Enter credentials.	Valid Credentials.			
			Click on the "Login" Button.				
SuccessAdminRun_AllValidInputs001	Admin logs in successfully and performs CRUD operations on shipment data.	Login Success	Navigate to the Login Page.	No User Logged in.	Admin is redirected to Admin Dashboard.	Admin is redirected to Admin Dashboard.	Pass
			Enter credentials.	Valid Credentials.			
			Click on the "Login" Button.				
SuccessAdminRun_AllValidInputs002	Admin logs in successfully and performs CRUD operations on shipment data.	Add a shipment	Target the Name field and send keys.	Logged in as Admin.	Shipment is added successfully.	Shipment is added successfully.	Pass
			Target the Type field and send keys.	Valid Data.			
			Target the Address field and send keys.				
			Click on Add Shipment Button.				
SuccessAdminRun_AllValidInputs003	Admin logs in successfully and performs CRUD operations on shipment data.	Edit a shipment	Click on the Edit button of a Shipment Data.	Logged in as Admin.	Shipment is updated successfully.	Shipment is updated successfully.	Pass
			Clear the Name.	Valid Data.			
			Update The name Field with new data.	Pre Existing Test Shipment data			
			Click on Update Shipment Button.				
SuccessAdminRun_AllValidInputs004	Admin logs in successfully and performs CRUD operations on shipment data.	Delete a Shipment	Click on the Delete button of a Shipment Data.	Logged in as Admin.	Shipment is deleted successfully.	Shipment is deleted successfully.	Pass
				Pre Existing Test Shipment data			

SuccessAdminRun_AllValidInputs005	Admin logs in successfully and performs CRUD operations on shipment data.	Log Out Success	Click on the Logout Button.	Logged in as Admin.	Redirected to login Page.	Redirected to login Page.	Pass
AddShipment_SubmitIncompleteData	Admin tries to add a data with incomplete fields.	Addition Failure due to Bad Request.	Target the Name field and send keys.	Logged in as Admin.	Admin is alerted that all fields are necessary.	Admin is alerted that all fields are necessary.	Pass
			Target the Type field and send keys.	Incomplete Data.			
			Click on Add Shipment Button.				
SearchBarTest_ByType	Admin tries to filter the data by type.	Search by Type	Target the search bar and send in the Type of a shipment data.	Logged in as Admin.	The data that matched the Type is the only data that is being displayed.	The data that matched the Type is the only data that is being displayed.	Pass
				Valid Data.			
				Valid Type.			
				Pre Existing Test Shipment data			
SearchBarTest_ByAddress	Admin tries to filter the data by address.	Search by Address	Target the search bar and send in the Address of a shipment data.	Logged in as Admin.	The data that matched the Address is the only data that is being displayed.	The data that matched the Address is the only data that is being displayed.	Pass
				Valid Data.			
				Valid Address.			
				Pre Existing Test Shipment data			
SearchBarTest_ByName	Admin tries to filter the data by name.	Search by Name	Target the search bar and send in the name of a shipment data.	Logged in as Admin.	The data that matched the name is the only data that is being displayed.	The data that matched the name is the only data that is being displayed.	Pass
				Valid Data.			
				Valid Name.			
				Pre Existing Test Shipment data			

```

using OpenQA.Selenium;
using OpenQA.Selenium.Chrome;

class Program
{
    public static void Main (string[] args)
    {
        //SetUp
        IWebDriver driver = new ChromeDriver();
        driver.Manage().Window.Maximize();

        SubmitEmptyRegisterFom(driver);

        SubmitEmptyLoginForm(driver);

        SubmitInvalidCredentials(driver);

        LoginAsAdmin(driver);

        SuccessAdminRun_AllValidInputs(driver);

        AddShipment_SubmitIncompleteData(driver);

        SearchBarTest_ByName(driver);
    }
}

```

```
SearchBarTest_ByType(driver);
```

```
SearchBarTest_ByAddress(driver);
```

```
EditShipment_SubmitIncompleteData(driver);
```

```
    //TearDown.  
    driver.Close();  
    driver.Quit();  
}
```

```
static void SubmitEmptyRegisterFom(IWebDriver driver)  
{  
    try  
    {  
        //Start from Register Page.  
        driver.Navigate().GoToUrl("http://localhost:3000/register");  
        Thread.Sleep(2000);
```

```
        //Submit Empty Register Form.  
        driver.FindElement(By.XPath("//button[text()='Register']")).Click();  
        Thread.Sleep(2000);  
        IAlert alert = driver.SwitchTo().Alert();  
        alert.Accept();  
        Thread.Sleep(2000);  
    }  
    catch (Exception ex)  
    {  
        Console.WriteLine(ex.Message);  
    }  
}
```

```
static void SubmitEmptyLoginForm(IWebDriver driver)  
{  
    try  
    {  
        //Start from Login Page.  
        driver.Navigate().GoToUrl("http://localhost:3000/");  
        Thread.Sleep(2000);
```

```
        //Submit Empty Login Form.  
        driver.FindElement(By.XPath("//button[text()='Login']")).Click();  
        Thread.Sleep(2000);  
        IAlert alert = driver.SwitchTo().Alert();  
        alert.Accept();  
        Thread.Sleep(2000);  
    } catch (Exception ex)  
    {  
        Console.WriteLine(ex.Message);  
    }  
}
```

```

static void SubmitInvalidCredentials(IWebDriver driver)
{
    try
    {
        //Start from Login Page.
        driver.Navigate().GoToUrl("http://localhost:3000/");
        Thread.Sleep(2000);

        //Submit Invalid Credentials.

        driver.FindElement(By.XPath("//input[@placeholder='Username']")).SendKeys("Someone Else");

        driver.FindElement(By.XPath("//input[@placeholder='Password']")).SendKeys("Somebody @123");
        driver.FindElement(By.XPath("//button[text()='Login']")).Click();
        Thread.Sleep(2000);
        IAlert alert = driver.SwitchTo().Alert();
        alert.Accept();
        Thread.Sleep(2000);
    }
    catch (Exception ex) {
        Console.WriteLine(ex.Message);
    }
}

static void LoginAsAdmin(IWebDriver driver)
{
    try
    {
        //Start from Login Page.
        driver.Navigate().GoToUrl("http://localhost:3000/");
        Thread.Sleep(2000);

        //Login as an "User" User.

        driver.FindElement(By.XPath("//input[@placeholder='Username']")).SendKeys("Admin");

        driver.FindElement(By.XPath("//input[@placeholder='Password']")).SendKeys("Admin@123");
        driver.FindElement(By.XPath("//button[text()='Login']")).Click();
        Thread.Sleep(2000);
    }
    catch (Exception ex)
    {
        Console.WriteLine(ex.Message);
    }
}

static void SuccessAdminRun_AllValidInputs(IWebDriver driver)

```

```

{
    try
    {
        LoginAsAdmin(driver);

        //Add a shipment.
        driver.FindElement(By.XPath("//input[@placeholder='Name']")).SendKeys("Box
A");

        driver.FindElement(By.XPath("//input[@placeholder='Type']")).SendKeys("Games");
        driver.FindElement(By.XPath("//input[@placeholder='Address']")).SendKeys("123
Avenue");
        driver.FindElement(By.XPath("//button[text()='Add Shipment']")).Click();
        Thread.Sleep(2000);

        //Edit a shipment.
        driver.FindElement(By.XPath("//button[text()='Edit']")).Click();
        Thread.Sleep(2000);
        driver.FindElement(By.XPath("//input[@placeholder='Name']")).Clear();

        driver.FindElement(By.XPath("//input[@placeholder='Name']")).SendKeys("Updated Box
A");
        driver.FindElement(By.XPath("//button[text()='Update Shipment']")).Click();
        Thread.Sleep(2000);

        //Delete a shipment.
        driver.FindElement(By.XPath("//button[text()='Delete']")).Click();
        Thread.Sleep(2000);

        //Logout.
        driver.FindElement(By.XPath("//button[text()='Logout']")).Click();
        Thread.Sleep(2000);
    }
    catch (Exception ex)
    {
        Console.WriteLine(ex.Message);
    }
}

static void AddShipment_SubmitIncompleteData(IWebDriver driver)
{
    try
    {
        LoginAsAdmin(driver);

        //Add a shipment.
        driver.FindElement(By.XPath("//input[@placeholder='Name']")).SendKeys("Box
A");

        driver.FindElement(By.XPath("//input[@placeholder='Type']")).SendKeys("Games");
        driver.FindElement(By.XPath("//button[text()='Add Shipment']")).Click();

```

```

        Thread.Sleep(2000);
        IAlert alert = driver.SwitchTo().Alert();
        alert.Accept();
        Thread.Sleep(2000);
    }
    catch (Exception ex)
    {
        Console.WriteLine(ex.Message);
    }
}

```

```

static void SearchBarTest_ByName(IWebDriver driver)
{
    try
    {
        LoginAsAdmin(driver);
    }
    catch { }
}

```

```

//Add a shipment.
driver.FindElement(By.XPath("//input[@placeholder='Name']")).SendKeys("Box
A");

driver.FindElement(By.XPath("//input[@placeholder='Type']")).SendKeys("Games");
driver.FindElement(By.XPath("//input[@placeholder='Address']")).SendKeys("123
Avenue");
driver.FindElement(By.XPath("//button[text()='Add Shipment']")).Click();
Thread.Sleep(2000);

```

```

//Add a shipment.
driver.FindElement(By.XPath("//input[@placeholder='Name']")).SendKeys("Box
B");

driver.FindElement(By.XPath("//input[@placeholder='Type']")).SendKeys("Movies");
driver.FindElement(By.XPath("//input[@placeholder='Address']")).SendKeys("456
Avenue");
driver.FindElement(By.XPath("//button[text()='Add Shipment']")).Click();
Thread.Sleep(2000);

```

```

//Send keys to search bar and delete.
driver.FindElement(By.XPath("//input[@placeholder='Search by name, type, or
address']")).SendKeys("Box A");
Thread.Sleep(2000);
driver.FindElement(By.XPath("//button[text()='Delete']")).Click();
Thread.Sleep(2000);

```

```

//Send keys to search bar and delete.
driver.FindElement(By.XPath("//input[@placeholder='Search by name, type, or
address']")).Clear();
driver.FindElement(By.XPath("//input[@placeholder='Search by name, type, or
address']")).SendKeys("Box B");
Thread.Sleep(2000);
driver.FindElement(By.XPath("//button[text()='Delete']")).Click();

```

```

        Thread.Sleep(2000);
    }
    catch (Exception ex)
    {
        Console.WriteLine(ex.Message);
    }
}

```

```

static void SearchBarTest_ByType(IWebDriver driver)
{
    try
    {
        LoginAsAdmin(driver);
    }
}

```

```

        //Add a shipment.
        driver.FindElement(By.XPath("//input[@placeholder='Name']")).SendKeys("Box
A");

driver.FindElement(By.XPath("//input[@placeholder='Type']")).SendKeys("Games");
driver.FindElement(By.XPath("//input[@placeholder='Address']")).SendKeys("123
Avenue");
driver.FindElement(By.XPath("//button[text()='Add Shipment']")).Click();
Thread.Sleep(2000);

```

```

        //Add a shipment.
        driver.FindElement(By.XPath("//input[@placeholder='Name']")).SendKeys("Box
B");

driver.FindElement(By.XPath("//input[@placeholder='Type']")).SendKeys("Movies");
driver.FindElement(By.XPath("//input[@placeholder='Address']")).SendKeys("456
Avenue");
driver.FindElement(By.XPath("//button[text()='Add Shipment']")).Click();
Thread.Sleep(2000);

```

```

        //Send keys to search bar and delete.
        driver.FindElement(By.XPath("//input[@placeholder='Search by name, type, or
address']")).SendKeys("Games");
        Thread.Sleep(2000);
        driver.FindElement(By.XPath("//button[text()='Delete']")).Click();
        Thread.Sleep(2000);

```

```

        //Send keys to search bar and delete.
        driver.FindElement(By.XPath("//input[@placeholder='Search by name, type, or
address']")).Clear();
        driver.FindElement(By.XPath("//input[@placeholder='Search by name, type, or
address']")).SendKeys("Movies");
        Thread.Sleep(2000);
        driver.FindElement(By.XPath("//button[text()='Delete']")).Click();
        Thread.Sleep(2000);
    }
    catch (Exception ex)
    {
        Console.WriteLine(ex.Message);
    }
}

```

```

    {
        Console.WriteLine(ex.Message);
    }
}

static void SearchBarTest_ByAddress(IWebDriver driver)
{
    try
    {
        LoginAsAdmin(driver);

        //Add a shipment.
        driver.FindElement(By.XPath("//input[@placeholder='Name']")).SendKeys("Box
A");

        driver.FindElement(By.XPath("//input[@placeholder='Type']")).SendKeys("Games");
        driver.FindElement(By.XPath("//input[@placeholder='Address']")).SendKeys("123
Avenue");
        driver.FindElement(By.XPath("//button[text()='Add Shipment']")).Click();
        Thread.Sleep(2000);

        //Add a shipment.
        driver.FindElement(By.XPath("//input[@placeholder='Name']")).SendKeys("Box
B");

        driver.FindElement(By.XPath("//input[@placeholder='Type']")).SendKeys("Movies");
        driver.FindElement(By.XPath("//input[@placeholder='Address']")).SendKeys("456
Avenue");
        driver.FindElement(By.XPath("//button[text()='Add Shipment']")).Click();
        Thread.Sleep(2000);

        //Send keys to search bar and delete.
        driver.FindElement(By.XPath("//input[@placeholder='Search by name, type, or
address']")).SendKeys("123");
        Thread.Sleep(2000);
        driver.FindElement(By.XPath("//button[text()='Delete']")).Click();
        Thread.Sleep(2000);

        //Send keys to search bar and delete.
        driver.FindElement(By.XPath("//input[@placeholder='Search by name, type, or
address']")).Clear();
        driver.FindElement(By.XPath("//input[@placeholder='Search by name, type, or
address']")).SendKeys("456");
        Thread.Sleep(2000);
        driver.FindElement(By.XPath("//button[text()='Delete']")).Click();
        Thread.Sleep(2000);
    }
    catch (Exception ex)
    {
        Console.WriteLine(ex.Message);
    }
}

```



```

    }

    static void EditShipment_SubmitIncompleteData(IWebDriver driver)
    {
        try
        {
            LoginAsAdmin(driver);

            //Add a shipment.
            driver.FindElement(By.XPath("//input[@placeholder='Name']")).SendKeys("Box
A");

            driver.FindElement(By.XPath("//input[@placeholder='Type']")).SendKeys("Games");
            driver.FindElement(By.XPath("//input[@placeholder='Address']")).SendKeys("123
Avenue");
            driver.FindElement(By.XPath("//button[text()='Add Shipment']")).Click();
            Thread.Sleep(2000);

            //Edit a shipment.
            driver.FindElement(By.XPath("//button[text()='Edit']")).Click();
            Thread.Sleep(2000);
            driver.FindElement(By.XPath("//input[@placeholder='Name']")).Clear();
            Thread.Sleep(2000);
            driver.FindElement(By.XPath("//button[text()='Update Shipment']")).Click();
            Thread.Sleep(2000);
            IAlert alert = driver.SwitchTo().Alert();
            alert.Accept();
            Thread.Sleep(2000);
        }
        catch (Exception ex)
        {
            Console.WriteLine(ex.Message);
        }
    }
}

```

UserRoleSeleniumTester

Project name :	Online Courier Management System
Module Name :	User Role User / User Dashboard
Reference Document :	UserRoleSeleniumTester
Created by :	Nisanth Saravanan
Date of creation :	13-08-2025
Date of review :	

SubmitEmptyRegisterForm	User tries to submit an empty register form.	Registration Validations	Navigate to Register Page	No Logged in User.	User is alerted that all fields are necessary.	User is alerted that all fields are necessary.	Pass
			Click on the "Register" Button.				
SubmitEmptyLoginForm	User tries to submit an empty register form.	Registration Validations	Navigate to Login Page	No Logged in User.	User is alerted that all fields are necessary.	User is alerted that all fields are necessary.	Pass
			Click on the "Login" Button.				
SubmitInvalidCredentials	User tried to login but credentials are invalid.	Login Failure due to Credentials.	Navigate to the Login Page.	No User Logged in.	User is alerted that credentials are invalid.	User is alerted that credentials are invalid.	Pass
			Enter credentials.	Invalid Credentials.			
			Click on the "Login" Button.				
LoginAsUser	User logs in successfully.	Login Success	Navigate to the Login Page.	No User Logged in.	User is redirected to User Dashboard.	User is redirected to User Dashboard.	Pass
			Enter credentials.	Valid Credentials.			
			Click on the "Login" Button.				
SearchBarTest_ByName	User tries to filter the data by name.	Search by Name	Target the search bar and send in the name of a shipment data.	Logged in as User.	The data that matched the name is the only data that is being displayed.	The data that matched the name is the only data that is being displayed.	Pass
				Valid Data.			
				Valid Name.			
				Pre Existing Test Shipment data			
SearchBarTest_ByType	User tries to filter the data by type.	Search by Type	Target the search bar and send in the Type of a shipment data.	Logged in as User.	The data that matched the Type is the only data that is being displayed.	The data that matched the Type is the only data that is being displayed.	Pass
				Valid Data.			
				Valid Type.			
				Pre Existing Test Shipment data			
SearchBarTest_ByAddress	User tries to filter the data by address.	Search by Address	Target the search bar and send in the Address of a shipment data.	Logged in as User.	The data that matched the Address is the only data that is being displayed.	The data that matched the Address is the only data that is being displayed.	Pass
				Valid Data.			
				Valid Address.			
				Pre Existing Test Shipment data			

```
using OpenQA.Selenium;
using OpenQA.Selenium.Chrome;
```

```
class Program
{
    public static void Main(string[] args)
```

```

{
    //SetUp
    IWebDriver driver = new ChromeDriver();
    driver.Manage().Window.Maximize();

    //Admin Side Preset.
    try
    {
        //Start from Login Page.
        driver.Navigate().GoToUrl("http://localhost:3000/");
        Thread.Sleep(2000);

        //Login as an "Admin" User.

        driver.FindElement(By.XPath("//input[@placeholder='Username']")).SendKeys("Admin");

        driver.FindElement(By.XPath("//input[@placeholder='Password']")).SendKeys("Admin@123");
        driver.FindElement(By.XPath("//button[text()='Login']")).Click();
        Thread.Sleep(2000);

        //Add a shipment.
        driver.FindElement(By.XPath("//input[@placeholder='Name']")).SendKeys("Box A");

        driver.FindElement(By.XPath("//input[@placeholder='Type']")).SendKeys("Games");
        driver.FindElement(By.XPath("//input[@placeholder='Address']")).SendKeys("123 Avenue");
        driver.FindElement(By.XPath("//button[text()='Add Shipment']")).Click();
        Thread.Sleep(2000);

        //Add a shipment.
        driver.FindElement(By.XPath("//input[@placeholder='Name']")).SendKeys("Box B");

        driver.FindElement(By.XPath("//input[@placeholder='Type']")).SendKeys("Movies");
        driver.FindElement(By.XPath("//input[@placeholder='Address']")).SendKeys("456 Avenue");
        driver.FindElement(By.XPath("//button[text()='Add Shipment']")).Click();
        Thread.Sleep(2000);
    }
    catch (Exception ex)
    {
        Console.WriteLine(ex.Message);
    }

    SubmitEmptyRegisterForm(driver);

    RegisterAsUser(driver);

    SubmitEmptyLoginForm(driver);
}

```

```
SubmitInvalidCredentials(driver);
```

```
LoginAsUser(driver);
```

```
SearchBarTest_ByName(driver);
```

```
SearchBarTest_ByType(driver);
```

```
SearchBarTest_ByAddress(driver);
```

```
//TearDown.
```

```
try
```

```
{
```

```
//End from Login Page.
```

```
driver.Navigate().GoToUrl("http://localhost:3000/");
```

```
Thread.Sleep(2000);
```

```
//Login as an "Admin" User.
```

```
driver.FindElement(By.XPath("//input[@placeholder='Username']")).SendKeys("Admin");
```

```
driver.FindElement(By.XPath("//input[@placeholder='Password']")).SendKeys("Admin@123");
```

```
driver.FindElement(By.XPath("//button[text()='Login']")).Click();
```

```
Thread.Sleep(2000);
```

```
//Delete sample data.
```

```
driver.FindElement(By.XPath("//button[text()='Delete']")).Click();
```

```
Thread.Sleep(2000);
```

```
driver.FindElement(By.XPath("//button[text()='Delete']")).Click();
```

```
Thread.Sleep(2000);
```

```
}
```

```
catch (Exception ex)
```

```
{
```

```
Console.WriteLine(ex.Message);
```

```
}
```

```
driver.Close();
```

```
driver.Quit();
```

```
}
```

```
static void SubmitEmptyRegisterFom(IWebDriver driver)
```

```
{
```

```
try
```

```
{
```

```
//Start from Register Page.
```

```
driver.Navigate().GoToUrl("http://localhost:3000/register");
```

```
Thread.Sleep(2000);
```

```
//Submit Empty Register Form.
```

```

        driver.FindElement(By.XPath("//button[text()='Register']")).Click();
        Thread.Sleep(2000);
        IAlert alert = driver.SwitchTo().Alert();
        alert.Accept();
        Thread.Sleep(2000);
    }
    catch (Exception ex)
    {
        Console.WriteLine(ex.Message);
    }
}

```

```

static void RegisterAsUser(IWebDriver driver)
{
    try
    {
        //Start from Register Page.
        driver.Navigate().GoToUrl("http://localhost:3000/register");
        Thread.Sleep(2000);

```

```

        //Submit Valid Creadentials.

```

```

driver.FindElement(By.XPath("//input[@placeholder='Username']")).SendKeys("Someone"
);

```

```

driver.FindElement(By.XPath("//input[@placeholder='Password']")).SendKeys("Somebody
@123");
        driver.FindElement(By.XPath("//button[text()='Register']")).Click();
        Thread.Sleep(2000);
        IAlert alert = driver.SwitchTo().Alert();
        alert.Accept();
        Thread.Sleep(2000);
    }
    catch (Exception ex)
    {
        Console.WriteLine(ex.Message);
    }
}

```

```

static void SubmitEmptyLoginForm(IWebDriver driver)
{
    try
    {
        //Start from Login Page.
        driver.Navigate().GoToUrl("http://localhost:3000/");
        Thread.Sleep(2000);

```

```

        //Submit Empty Login Form.
        driver.FindElement(By.XPath("//button[text()='Login']")).Click();
        Thread.Sleep(2000);
        IAlert alert = driver.SwitchTo().Alert();

```

```

        alert.Accept();
        Thread.Sleep(2000);
    }
    catch (Exception ex)
    {
        Console.WriteLine(ex.Message);
    }
}

```

```

static void SubmitInvalidCredentials(IWebDriver driver)
{
    try
    {
        //Start from Login Page.
        driver.Navigate().GoToUrl("http://localhost:3000/");
        Thread.Sleep(2000);

```

```

        //Submit Invalid Credentials.

```

```

driver.FindElement(By.XPath("//input[@placeholder='Username']")).SendKeys("Someone Else");

```

```

driver.FindElement(By.XPath("//input[@placeholder='Password']")).SendKeys("Somebody @123");
    driver.FindElement(By.XPath("//button[text()='Login']")).Click();
    Thread.Sleep(2000);
    IAlert alert = driver.SwitchTo().Alert();
    alert.Accept();
    Thread.Sleep(2000);
}
catch(Exception ex)
{
    Console.WriteLine(ex.Message);
}
}

```

```

static void LoginAsUser(IWebDriver driver)
{
    try
    {
        //Start from Login Page.
        driver.Navigate().GoToUrl("http://localhost:3000/");
        Thread.Sleep(2000);

```

```

        //Login as an "User" User.

```

```

driver.FindElement(By.XPath("//input[@placeholder='Username']")).SendKeys("User306");

```

```

driver.FindElement(By.XPath("//input[@placeholder='Password']")).SendKeys("User@123");

```

```

        driver.FindElement(By.XPath("//button[text()='Login']")).Click();
        Thread.Sleep(2000);
    }
    catch (Exception ex)
    {
        Console.WriteLine(ex.Message);
    }
}

```

```

static void SearchBarTest_ByName(IWebDriver driver)
{
    try
    {
        LoginAsUser(driver);

```

```

        //Send keys to search bar.
        driver.FindElement(By.XPath("//input[@placeholder='Search by name, type, or address']")).SendKeys("Box A");
        Thread.Sleep(2000);

```

```

        //Send keys to search bar.
        driver.FindElement(By.XPath("//input[@placeholder='Search by name, type, or address']")).Clear();
        driver.FindElement(By.XPath("//input[@placeholder='Search by name, type, or address']")).SendKeys("Box B");
        Thread.Sleep(2000);
    }
    catch (Exception ex)
    {
        Console.WriteLine(ex.Message);
    }
}

```

```

static void SearchBarTest_ByType(IWebDriver driver)
{
    try
    {
        LoginAsUser(driver);

```

```

        //Send keys to search bar.
        driver.FindElement(By.XPath("//input[@placeholder='Search by name, type, or address']")).SendKeys("Games");
        Thread.Sleep(2000);

```

```

        //Send keys to search bar.
        driver.FindElement(By.XPath("//input[@placeholder='Search by name, type, or address']")).Clear();
        driver.FindElement(By.XPath("//input[@placeholder='Search by name, type, or address']")).SendKeys("Movies");
        Thread.Sleep(2000);
    }
}

```

```
    catch(Exception ex)
    {
        Console.WriteLine(ex.Message);
    }
}
```

```
static void SearchBarTest_ByAddress(IWebDriver driver)
{
    try
    {
        LoginAsUser(driver);
```

```
        //Send keys to search bar.
        driver.FindElement(By.XPath("//input[@placeholder='Search by name, type, or address']")).SendKeys("123");
        Thread.Sleep(2000);
```

```
        //Send keys to search bar.
        driver.FindElement(By.XPath("//input[@placeholder='Search by name, type, or address']")).Clear();
        driver.FindElement(By.XPath("//input[@placeholder='Search by name, type, or address']")).SendKeys("456");
        Thread.Sleep(2000);
    }
    catch (Exception ex)
    {
        Console.WriteLine(ex.Message);
    }
}
```


RegisterAndLoginSeleniumTester

Project name :	Online Courier Management System
Module Name :	Register and Login
Reference Document :	RegisterAndLoginSeleniumTester
Created by :	Nisanth Saravanan
Date of creation :	13-08-2025
Date of review :	

SubmitEmptyRegisterForm	User tries to submit an empty register form.	Registration Validations	Navigate to Register Page	No Logged in User.	User is alerted that all fields are necessary.	User is alerted that all fields are necessary.	Pass
			Click on the "Register" Button.				
SubmitEmptyLoginForm	User tries to submit an empty register form.	Registration Validations	Navigate to Login Page	No Logged in User.	User is alerted that all fields are necessary.	User is alerted that all fields are necessary.	Pass
			Click on the "Login" Button.				
SubmitInvalidCredentials	User tried to login but credentials are invalid.	Login Failure due to Credentials.	Navigate to the Login Page.	No User Logged in.	User is alerted that credentials are invalid.	User is alerted that credentials are invalid.	Pass
			Enter credentials.	Invalid Credentials.			
			Click on the "Login" Button.				
LoginAsUser	User logs in successfully.	Login Success	Navigate to the Login Page.	No User Logged in.	User is redirected to User Dashboard.	User is redirected to User Dashboard.	Pass
			Enter credentials.	Valid Credentials.			
			Click on the "Login" Button.				
LoginAsAdmin	Admin logs in successfully.	Login Success	Navigate to the Login Page.	No User Logged in.	Admin is redirected to Admin Dashboard.	Admin is redirected to Admin Dashboard.	Pass
			Enter credentials.	Valid Credentials.			
			Click on the "Login" Button.				
RegisterAsUser	User tries to register into new account.	Registration Failure.	Navigate to the Register Page.	No User Logged in.	User is alerted of their successful registration.	User is alerted of their successful registration.	Pass
			Enter credentials.	Valid Credentials.			
			Click on the "Register" Button.				
FailUserRegister	User tries to register into new account.	Registration Success.	Navigate to the Register Page.	No User Logged in.	Users is alerted that the credentials are invalid.	Users is alerted that the credentials are invalid.	Pass
			Enter credentials.	Invalid Pre-existing Credentials.			
			Click on the "Register" Button.				
LoginAsUser0.5	Admin tries to logout.	Log Out Success	Click on the Logout Button.	Logged in as Admin.	Redirected to login Page.	Redirected to login Page.	Pass

LoginAsAdmin0.5	User tries to logout.	Log Out Success	Click on the Logout Button.	Logged in as User.	Redirected to login Page.	Redirected to login Page.	Pass

```

using OpenQA.Selenium;
using OpenQA.Selenium.Chrome;

class Program
{
    public static void Main(string[] args)
    {
        //SetUp
        IWebDriver driver = new ChromeDriver();
        driver.Manage().Window.Maximize();

        SubmitEmptyRegisterFom(driver);

        SubmitEmptyLoginForm(driver);

        SubmitInvalidCredentials(driver);

        RegisterAsUser(driver);

        FailUserRegister(driver);

        LoginAsAdmin(driver);

        LoginAsUser(driver);

        //TearDown.
        driver.Close();
        driver.Quit();
    }

    static void SubmitEmptyRegisterFom(IWebDriver driver)
    {
        try
        {
            //Start from Register Page.
            driver.Navigate().GoToUrl("http://localhost:3000/register");
            Thread.Sleep(2000);

            //Submit Empty Register Form.
            driver.FindElement(By.XPath("//button[text()='Register']")).Click();
            Thread.Sleep(2000);
            IAlert alert = driver.SwitchTo().Alert();
            alert.Accept();
            Thread.Sleep(2000);
        }
        catch { }
    }
}

```

```

    }
    catch (Exception ex)
    {
        Console.WriteLine(ex.Message);
    }
}

```

```

static void SubmitEmptyLoginForm(IWebDriver driver)
{
    try
    {
        //Start from Login Page.
        driver.Navigate().GoToUrl("http://localhost:3000/");
        Thread.Sleep(2000);

```

```

        //Submit Empty Login Form.
        driver.FindElement(By.XPath("//button[text()='Login']")).Click();
        Thread.Sleep(2000);
        IAlert alert = driver.SwitchTo().Alert();
        alert.Accept();
        Thread.Sleep(2000);
    }
    catch (Exception ex)
    {
        Console.WriteLine(ex.Message);
    }
}

```

```

static void SubmitInvalidCredentials(IWebDriver driver)
{
    try
    {
        //Start from Login Page.
        driver.Navigate().GoToUrl("http://localhost:3000/");
        Thread.Sleep(2000);

```

```

        //Submit Invalid Credentials.

```

```

driver.FindElement(By.XPath("//input[@placeholder='Username']")).SendKeys("Someone
else");

```

```

driver.FindElement(By.XPath("//input[@placeholder='Password']")).SendKeys("Somebody
@123");
    driver.FindElement(By.XPath("//button[text()='Login']")).Click();
    Thread.Sleep(2000);
    IAlert alert = driver.SwitchTo().Alert();
    alert.Accept();
    Thread.Sleep(2000);
}
catch (Exception ex)
{

```

```
        Console.WriteLine(ex.Message);
    }
}
```

```
static void RegisterAsUser(IWebDriver driver)
{
    try
    {
        //Start from Register Page.
        driver.Navigate().GoToUrl("http://localhost:3000/register");
        Thread.Sleep(2000);
```

```
        //Submit Valid Credentials.
```

```
driver.FindElement(By.XPath("//input[@placeholder='Username']")).SendKeys("Someone in the wind.");
```

```
driver.FindElement(By.XPath("//input[@placeholder='Password']")).SendKeys("Somebody @123");
```

```
    driver.FindElement(By.XPath("//button[text()='Register']")).Click();
    Thread.Sleep(2000);
    IAlert alert = driver.SwitchTo().Alert();
    alert.Accept();
    Thread.Sleep(2000);
```

```
    }
    catch(Exception ex)
    {
        Console.WriteLine(ex.Message);
    }
}
```

```
static void FailUserRegister(IWebDriver driver)
{
    try
```

```
    {
        //Start from Register Page.
        driver.Navigate().GoToUrl("http://localhost:3000/register");
        Thread.Sleep(2000);
```

```
        //Submit Valid Credentials.
```

```
driver.FindElement(By.XPath("//input[@placeholder='Username']")).SendKeys("Someone in the wind.");
```

```
driver.FindElement(By.XPath("//input[@placeholder='Password']")).SendKeys("Somebody @123");
```

```
    driver.FindElement(By.XPath("//button[text()='Register']")).Click();
    Thread.Sleep(2000);
    IAlert alert = driver.SwitchTo().Alert();
    alert.Accept();
    Thread.Sleep(2000);
```

```

    }
    catch(Exception ex)
    {
        Console.WriteLine(ex.Message);
    }
}

static void LoginAsAdmin(IWebDriver driver)
{
    try
    {
        //Start from Login Page.
        driver.Navigate().GoToUrl("http://localhost:3000/");
        Thread.Sleep(2000);

        //Login as an "User" User.

driver.FindElement(By.XPath("//input[@placeholder='Username']")).SendKeys("Admin");

driver.FindElement(By.XPath("//input[@placeholder='Password']")).SendKeys("Admin@123");
        driver.FindElement(By.XPath("//button[text()='Login']")).Click();
        Thread.Sleep(2000);

        //Logout.
        driver.FindElement(By.XPath("//button[text()='Logout']")).Click();
        Thread.Sleep(2000);
    }
    catch(Exception ex)
    {
        Console.WriteLine(ex.Message);
    }
}

static void LoginAsUser(IWebDriver driver)
{
    try
    {
        //Start from Login Page.
        driver.Navigate().GoToUrl("http://localhost:3000/");
        Thread.Sleep(2000);

        //Login as an "User" User.

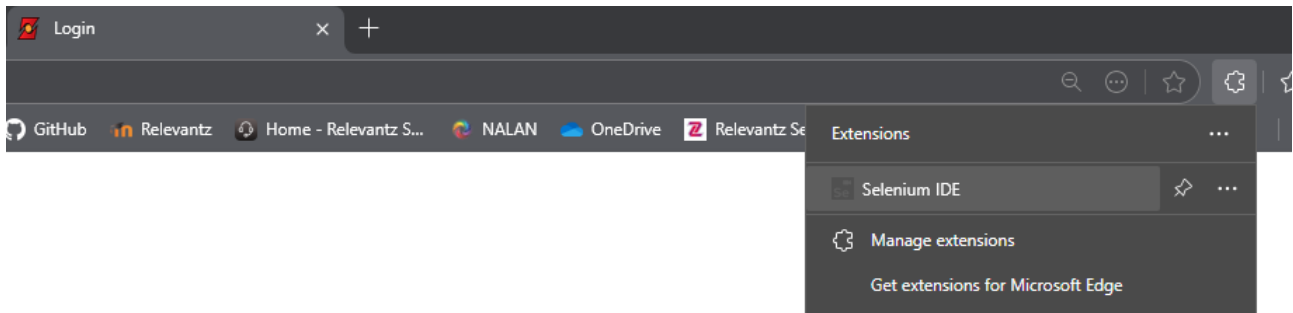
driver.FindElement(By.XPath("//input[@placeholder='Username']")).SendKeys("User306");

driver.FindElement(By.XPath("//input[@placeholder='Password']")).SendKeys("User@123");
        driver.FindElement(By.XPath("//button[text()='Login']")).Click();
        Thread.Sleep(2000);
    }
}

```

```
        //Logout.  
        driver.FindElement(By.XPath("//button[text()='Logout']")).Click();  
        Thread.Sleep(2000);  
    }  
    catch (Exception ex)  
    {  
        Console.WriteLine(ex.Message);  
    }  
}  
}
```

Selenium IDE



Login

Admin

Login

New User? Register.



Welcome to Selenium IDE!
Version 3.17.0

What would you like to do?

[Record a new test in a new project](#)

[Open an existing project](#)

[Create a new project](#)

[Close Selenium IDE](#)

To learn more on Selenium IDE and how to use it visit the [the Selenium IDE project page](#).

Name your new project

Please provide a name for your new project.

PROJECT NAME

Online Shipment Manager

You can change the name of your project at any time by clicking it and entering a new name.

OK

CANCEL

Set your project's base URL

Before you can start recording, you must specify a valid base URL for your project. Your tests will start by navigating to this URL.

BASE URL

START RECORDING

CANCEL

Login

Login

New User? Register.

 Selenium IDE is recording

Name your new test

Please provide a name for your new test.

TEST NAME

You can change it at any time by clicking the  icon next to its name in the tests panel.

OK

LATER

Project: Online Shipment Manager*

Tests ▾ + ▶ ⏮ ⏭ ⏴ ⏵ ⌚

Search tests... Run current test Ctrl+R

	Command	Target	Value
1	open	/	
2	set window size	739x720	
3	type	css=tr:nth-child(1) .form-control	Admin
4	type	css=tr:nth-child(2) .form-control	Admin@123
5	click	css=.container	
6	click	css=tr:nth-child(2) .form-control	
7	click	css=tr:nth-child(3) button	

Command # 

Target  

Value

Description

Log Reference

Selenium IDE - Online Shipment Manager*

Project: Online Shipment Manager*

Executing ▾ ▶ ⏮ ⏭ ⏴ ⏵ ⌚

Admin User Test Run* http://localhost:3000

	Command	Target	Value
16	mouse out	css=	
17	click	id=n	
18	type	id=n	
19	click	id=ty	
20	type	id=ty	
21	click	id=a	
22	type	id=a	

Command # 

Target  

Value

Description

Runs: 0 Failures: 0

Log Reference

16. mouseOut on css=button:nth-child(1) OK

17. click on id=name-input OK

18. type on id=name-input with value Box B OK

19. click on id=type-input OK

20. type on id=type-input with value Movies OK

21. click on id=address-input OK

22. type on id=address-input with value 456 Street

Admin Dash

localhost:3000/admin

Email - Nisanth Sara... Spent time - Details... GitHub Relevantz Other favourites

Admin Dashboard

Logout

Box B

Movies

Address

Add Shipment

Search by name, type, or address

Name: Box A Type: Games Addressed To: 123 Street Edit Delete

Windows Taskbar: 03:23 PM 13/08/2025

33. click on css=body OK 15:23:58

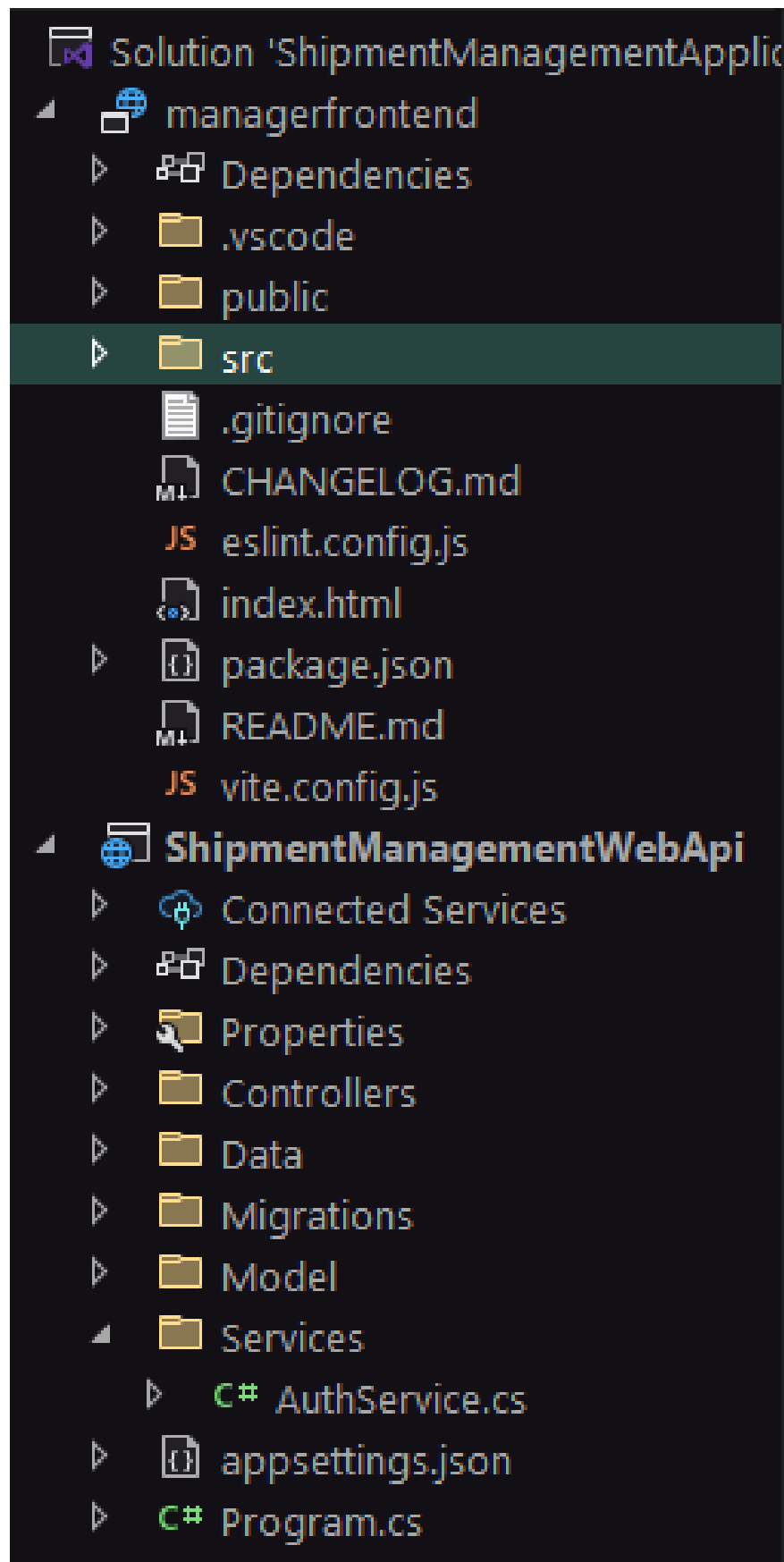
34. type on css=td > .form-control with value Movies OK 15:23:58

35. click on css=button:nth-child(5) OK 15:23:58

'Admin User Test Run' completed successfully 15:23:59

'Admin User Test Run' completed successfully

Code Base



Backend:

/Services/AuthService.cs

```
using ShipmentManagementWebApi.Data;
using ShipmentManagementWebApi.Models;
using Microsoft.EntityFrameworkCore;

namespace ShipmentManagementWebApi.Services;

public class AuthService
{
    private readonly AppDbContext _context;

    public AuthService(AppDbContext context)
    {
        _context = context;
    }

    public User Authenticate(string username, string password)
    {
        return _context.Users.FirstOrDefault(u => u.Username == username && u.Password
== password);
    }

    public bool Register(User user)
    {
        if (_context.Users.Any(u => u.Username == user.Username)) return false;
        _context.Users.Add(user);
        _context.SaveChanges();
        return true;
    }
}
```

appsettings.json

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "ConnectionStrings": {
    "DefaultConnection":
"Server=127.0.0.1;Port=3306;Database=ShipmentDb;User=root;Password=root"
  },
  "AllowedHosts": "*"
}
```

Program.cs

```
using Microsoft.EntityFrameworkCore;
using ShipmentManagementWebApi.Data;
using ShipmentManagementWebApi.Services;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.

builder.Services.AddControllers();
// Learn more about configuring Swagger/OpenAPI at
https://aka.ms/aspnetcore/swashbuckle
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

// Configure EF Core with MySQL
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseMySQL(
        builder.Configuration.GetConnectionString("DefaultConnection"),

ServerVersion.AutoDetect(builder.Configuration.GetConnectionString("DefaultConnection"
))
));

// Register custom services
builder.Services.AddScoped<AuthService>();

builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowReactApp",
        policy => policy
            .WithOrigins("http://localhost:51097") // React dev server
            .AllowAnyHeader()
            .AllowAnyMethod());
});

var app = builder.Build();

app.UseCors("AllowReactApp");

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();

app.UseAuthorization();

app.MapControllers();

app.Run();
```

/Models/User.cs

```
using System.ComponentModel.DataAnnotations;

namespace ShipmentManagementWebApi.Models
{
    public class User
    {
        [Key]
        public int Id { get; set; }
        public string? Username { get; set; }
        public string? Password { get; set; } // Plain text for simplicity
        public string? Role { get; set; } = "User"; // Default role
    }
}
```

/Models/Shipment.cs

```
using System.ComponentModel.DataAnnotations;

namespace ShipmentManagementWebApi.Models
{
    public class Shipment
    {
        [Key]
        public int Id { get; set; }
        public string? Name { get; set; }
        public string? Type { get; set; }
        public string? Address { get; set; }
    }
}
```

/Data/AppDbContext.cs

```
using Microsoft.EntityFrameworkCore;
using System.Collections.Generic;
using ShipmentManagementWebApi.Models;

namespace ShipmentManagementWebApi.Data
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options) : base(options)
        { }

        public DbSet<User> Users { get; set; }
        public DbSet<Shipment> Shipments { get; set; }
    }
}
```


/Controllers/AuthController.cs

```
using Microsoft.AspNetCore.Mvc;
using ShipmentManagementWebApi.Services;
using ShipmentManagementWebApi.Models;

[ApiController]
[Route("api/[controller]")]
public class AuthController : ControllerBase
{
    private readonly AuthService _authService;

    public AuthController(AuthService authService)
    {
        _authService = authService;
    }

    [HttpPost("register")]
    public IActionResult Register(User user)
    {
        user.Role = "User"; // Default role
        if (_authService.Register(user)) return Ok("Registered successfully");
        return BadRequest("Username already exists");
    }

    [HttpPost("login")]
    public IActionResult Login(User user)
    {
        var loggedInUser = _authService.Authenticate(user.Username, user.Password);
        if (loggedInUser == null) return Unauthorized("Invalid credentials");
        return Ok(loggedInUser); // Return user object with role
    }
}
```

/Controllers/ShipmentController.cs

```
using Microsoft.AspNetCore.Mvc;
using ShipmentManagementWebApi.Data;
using ShipmentManagementWebApi.Models;

[ApiController]
[Route("api/[controller]")]
public class ShipmentController : ControllerBase
{
    private readonly ApplicationDbContext _context;

    public ShipmentController(ApplicationDbContext context)
    {
        _context = context;
    }

    [HttpGet]
    public IActionResult GetAll() => Ok(_context.Shipments.ToList());

    [HttpGet("{id}")]
    public IActionResult Get(int id)
    {
        var shipment = _context.Shipments.Find(id);
        if (shipment == null) return NotFound();
        return Ok(shipment);
    }

    [HttpPost]
    public IActionResult Create(Shipment shipment, [FromQuery] string role)
    {
        if (role != "Admin") return Unauthorized("Only admin can create shipments");
        _context.Shipments.Add(shipment);
        _context.SaveChanges();
        return Ok(shipment);
    }

    [HttpPut("{id}")]
    public IActionResult Update(int id, Shipment updatedshipment, [FromQuery] string
role)
    {
        if (role != "Admin") return Unauthorized("Only admin can update shipments");
        var shipment = _context.Shipments.Find(id);
        if (shipment == null) return NotFound();

        shipment.Name = updatedshipment.Name;
        shipment.Type = updatedshipment.Type;
        shipment.Address = updatedshipment.Address;

        _context.SaveChanges();
        return Ok(shipment);
    }

    [HttpDelete("{id}")]
    public IActionResult Delete(int id, [FromQuery] string role)
    {
        if (role != "Admin") return Unauthorized("Only admin can delete shipments");
        var shipment = _context.Shipments.Find(id);
        if (shipment == null) return NotFound();

        _context.Shipments.Remove(shipment);
        _context.SaveChanges();
    }
}
```

```
        return Ok("Deleted");  
    }  
}
```

Frontend – React JS

/src/main.jsx

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';
import { AuthProvider } from './context/AuthContext';
import './index.css';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <AuthProvider>
    <App />
  </AuthProvider>
);
```

/src/App.jsx

```
import React, { useContext } from 'react';
import { BrowserRouter as Router, Routes, Route, Navigate } from 'react-router-dom';
import Login from './components/Login';
import Register from './components/Register';
import AdminDashboard from './components/AdminDashboard';
import UserDashboard from './components/UserDashboard';
import { AuthContext } from './context/AuthContext';
import './App.css';

function App() {
  const { user } = useContext(AuthContext);

  return (
    <Router>
      <Routes>
        <Route path="/" element={<Login />} />
        <Route path="/register" element={<Register />} />
        <Route path="/admin" element={user?.role === 'Admin' ? <AdminDashboard
/> : <Navigate to="/" />} />
        <Route path="/user" element={user?.role === 'User' ? <UserDashboard />
: <Navigate to="/" />} />
      </Routes>
    </Router>
  );
}

export default App;
```

/services/api.js

```
//api.js
import axios from 'axios';

const API = axios.create({
  baseURL: 'http://localhost:5099/api', // Adjust if needed
});

export default API;
```

/context/AuthContext.jsx

```
//AuthContext.jsx
import React, { createContext, useState } from 'react';

// eslint-disable-next-line react-refresh/only-export-components
export const AuthContext = createContext();

export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(null); // Holds logged-in user

  return (
    <AuthContext.Provider value={{ user, setUser }}>
      {children}
    </AuthContext.Provider>
  );
};
```

/components/Login.jsx

```
import React, { useState, useContext, useEffect } from 'react';
import { useNavigate } from 'react-router-dom';
import API from '../services/api';
import { AuthContext } from '../context/AuthContext';

function Login() {
  const [username, setUsername] = useState('');
  const [password, setPassword] = useState('');
  const { setUser } = useContext(AuthContext);
  const { user } = useContext(AuthContext);
  const navigate = useNavigate();

  const handleLogin = async () => {
    if (username == '' || password == '') {
      alert("All credentials are mandatory.");
      return;
    }

    try {
      const res = await API.post('/auth/login', { username, password });
      setUser(res.data);
      res.data.role === 'Admin' ? navigate('/admin') : navigate('/user');
    } catch {
      alert('Invalid credentials');
    }
  };

  const handleRegister = () => {
    navigate('/register');
  };

  useEffect(() => {
    document.title = "Login";

    if (user?.role == "Admin") {
      navigate("/admin");
    }
    else if (user?.role == "User") {
      navigate("/user")
    }
  });

  return (
    <>
      <div class="container">
        <h2>Login</h2>
        <table>
          <tr>
            <td>
              <div class="input-group input-group-lg mb-3">
                <input class="form-control" placeholder="Username"
onChange={e => setUsername(e.target.value)} />
              </div>
            </td>
          </tr>
          <tr>
            <td>
              <div class="input-group input-group-lg mb-3">
                <input class="form-control" placeholder="Password"
type="password" onChange={e => setPassword(e.target.value)} />
              </div>
            </td>
          </tr>
        </table>
      </div>
    </>
  );
}
```



```

        </div>
      </td>
    </tr>
    <tr>
      <td>
        <button onClick={handleLogin}>Login</button>
      </td>
    </tr>
    <br><br>
    <tr>
      <td>
        <button onClick={handleRegister}>New User?
Register.</button>
      </td>
    </tr>
  </table>
</div>
</>
);
}

export default Login;

```

/components/Register.jsx

```
import React, { useState, useEffect, useContext } from 'react';
import { useNavigate } from 'react-router-dom';
import API from '../services/api';
import { AuthContext } from '../context/AuthContext';

function Register() {
  const [username, setUsername] = useState('');
  const [password, setPassword] = useState('');
  const { user } = useContext(AuthContext);
  const navigate = useNavigate();

  const handleRegister = async () => {
    if (username == '' || password == '') {
      alert("All credentials are mandatory.");
      return;
    }

    try {
      await API.post('/auth/register', { username, password });
      alert('Registration successful');
      navigate("/");
    } catch {
      alert('Registration failed');
    }
  };

  const handleLogin = () => {
    navigate('/');
  };

  useEffect(() => {
    document.title = "Register";

    if (user?.role == "Admin") {
      navigate("/admin");
    }
    else if (user?.role == "User") {
      navigate("/user")
    }
  });

  return (
    <>
      <div class="container">
        <h2>Register</h2>
        <table>
          <tr>
            <td>
              <div class="input-group input-group-lg mb-3">
                <input class="form-control" placeholder="Username"
onChange={e => setUsername(e.target.value)} />
              </div>
            </td>
          </tr>
          <tr>
            <td>
              <div class="input-group input-group-lg mb-3">
                <input class="form-control" placeholder="Password"
type="password" onChange={e => setPassword(e.target.value)} />
              </div>
            </td>
          </tr>
        </table>
      </div>
    </>
  );
}
```

```
        </td>
      </tr>
      <tr>
        <td>
          <button onClick={handleRegister}>Register</button>
        </td>
      </tr>
      <br><br>
      <tr>
        <td>
          <button onClick={handleLogin}>Old User? Login.</button>
        </td>
      </tr>
    </table>
  </div>
</>
  );
}

export default Register;
```

/components/AdminDashboard.jsx

```
import React, { useContext, useEffect, useState } from 'react';
import API from '../services/api';
import { AuthContext } from '../context/AuthContext';

function AdminDashboard() {
  const { user } = useContext(AuthContext);
  const [shipments, setShipment] = useState([]);
  const { setUser } = useContext(AuthContext);
  const [searchTerm, setSearchTerm] = useState('');
  const [form, setForm] = useState({ name: '', type: '', address: '' });
  const [editingId, setEditingId] = useState(null);

  const fetchShipments = async () => {
    const res = await API.get('/shipment');
    setShipment(res.data);
  };

  const createOrUpdateShipment = async () => {
    if (form.name == '' || form.type == '' || form.address == '') {
      alert("All fields Needed!");
      return;
    }

    if (editingId) {
      await API.put(`/shipment/${editingId}?role=${user.role}`, form);
      setEditingId(null);
    } else {
      await API.post(`/shipment?role=${user.role}`, form);
    }
    setForm({ name: '', type: '', address: '' });
    fetchShipments();
  };

  const deleteShipment = async (id) => {
    await API.delete(`/shipment/${id}?role=${user.role}`);
    fetchShipments();
  };

  const startEdit = (shipment) => {
    setForm(shipment);
    setEditingId(shipment.id);
  };

  const filteredShipments = shipments.filter(s =>
    s.name.toLowerCase().includes(searchTerm.toLowerCase()) ||
    s.type.toLowerCase().includes(searchTerm.toLowerCase()) ||
    s.address.toString().includes(searchTerm)
  );

  const logout = () => {
    setUser(null);
  };

  useEffect(() => {
    document.title = "Admin Dash";

    fetchShipments();
  }, []);
}
```

[illegible]

```
    }}  
    </ul>  
  </div>  
);  
}  
  
export default AdminDashboard;
```

/components/UserDashboard.jsx

```
import React, { useContext, useEffect, useState } from 'react';
import API from '../services/api';
import { AuthContext } from '../context/AuthContext';

function UserDashboard() {
    const [shipments, setShipment] = useState([]);
    const [searchTerm, setSearchTerm] = useState('');
    const { setUser } = useContext(AuthContext);

    useEffect(() => {
        document.title = "User Dash";

        API.get('/shipment').then(res => setShipment(res.data));
    }, []);

    const logout = () => {
        setUser(null);
    };

    const filteredShipments = shipments.filter(s =>
        s.name.toLowerCase().includes(searchTerm.toLowerCase()) ||
        s.type.toLowerCase().includes(searchTerm.toLowerCase()) ||
        s.address.toString().includes(searchTerm)
    );

    return (
        <div class="container">
            <h2>User Dashboard</h2>
            <br/>
            <button onClick={logout} style={{ marginBottom: '1rem' }}>
                Logout
            </button>
            <br/>
            <br/>
            <table>
                <tr>
                    <td>
                        <input
                            class="form-control"
                            placeholder="Search by name, type, or address"
                            value={searchTerm}
                            onChange={e => setSearchTerm(e.target.value)}
                        />
                    </td>
                </tr>
            </table>
            <br/>
            <br/>
            <ul>
                {filteredShipments.map(s => (
                    <li key={s.id}>
                        <strong>Name:</strong> {s.name}, &nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&~
                        <strong>Type:</strong> {s.type}, &nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;& ~
                        <strong>Addressed To:</strong> {s.address} &nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&~
                    </li>
                ))}
            </ul>
        </div>
    );
}
```

```
export default UserDashboard;
```